Multi-Rail Isolated Power Supply Monitoring System

Capstone Design Documentation

Michael Capotosto

November 3, 2026

Rev. ET450-W8

Contents

1 Introduction **1**

1.1 Project Overview 1

1.2 Project Motivation 2

1.3 System Architecture 2

1.4 Design Objectives 2

1.5 Project Scope 3

1.6 Report Organization 4

2 Problem Background and Definition **5**

2.1 Purpose and Scope 5

2.2 Problem Definition 5

2.3 Proposed Solution 6

3 Technology Solution **7**

4 Chosen Conceptual Design **9**

4.0.1 Functional Block Diagram 10

5 Potential Social, Ethical, Legal, and Environmental Impacts of the Project **11**

5.1 Societal Impact 11

5.2 Global Impact 11

5.3 Legal and Regulatory Impact 12

5.4 Ethical Impact 12

5.5 Environmental Impact 13

5.6 Relationship to the Original Requirements 13

6 Design Methodology **15**

7 Simulation Results **17**

8 Final Design **19**

9 Solution Implementation **21**

9.1 Testing the LTC2945 21

9.1.1 Calculating Shunt Value for Test 22

9.1.2 Pictures of Evaluation Board, Display, and Phoebus Panels 23

9.2 Bring Up and Testing of the First Spin Board 27

9.3 Assembly Verification/Visual Inspection and Smoke Test 28

9.3.1 Visual Inspection 28

9.3.2 Smoke Test 29

9.4 I^2C SDA Communication Problem 29

9.4.1 Initial Symptoms 29

9.4.2 SDA Signal Troubleshooting 30

9.4.3 Identification of Incorrect Component 31

9.4.4 Inverter Bodge Modification 31

9.4.5 Verification After Modification 32

9.4.6 Further Bodges and Repairs 34

10 Evaluation of Solution's Efficacy **39**

10.1 Automated Functional Tests 39

10.1.1 System Under Test 39

10.1.2 Scope of Testing 40

10.1.3 Test Equipment and Software 41

10.1.4 Test Configuration 42

10.1.5 System Acceptance Criteria 42

10.1.6 System Test Matrix 44

10.1.7 Detailed Test Procedures 46

10.1.8 Automated Test Data Collection 49

10.1.9 Test Execution and Result Classification 50

10.1.10 Suspension and Resumption Criteria 50

10.1.11 Test Responsibilities 51
10.1.12 Testing Risks and Contingencies 51
10.1.13 Test Deliverables 52
10.1.14 Test Completion Criteria 52
10.1.15 Test Reports 53

11 Improvements and Optimizations to Solution 55

12 Conclusions 57
12.0.1 Did the Project Meet the Requirements of the Scope? 57
12.0.2 Was the Project Completed on Time? 57
12.0.3 Was the Project Completed Under Budget? 58
12.0.4 Did the Quality and Performance Meet Stakeholder Expectations? . . 58
12.0.5 Overall Project Assessment 59

13 References 61

A Functional Specifications 63

B Design Calculations and Circuit Analysis 65
B.1 Current-Sense Shunt Resistor Sizing 65
B.1.1 Current Measurement Resolution 66
B.1.2 Production Shunt Design Table 66
B.1.3 Evaluation-Board Shunt Calculation 66
B.2 PCB Trace and Copper-Pour Current Capacity 67
B.3 I^2C Pull-Up Resistor Sizing 67
B.4 LTC2945 Voltage Measurement Resolution 67
B.5 Negative-Rail Sign Convention 68
B.6 Alarm-Limit Scaling 68
B.7 Controller-Supply Power Budget 68
B.8 Component Capability Checks 69
B.9 Current-Sense Resistor Power Margin 69
B.10 LTC2945 Input and ADC Range 69
B.11 Microcontroller I/O Compatibility 70

B.12 Digital-Isolation Margin 70
B.13 3.3 V Regulator Loading 70
B.14 Tolerance and Measurement-Uncertainty Considerations 70
B.15 Reference Specifications Used for Capability Checks 71

C Design Documents **73**
C.1 Schematics 73
C.2 Software Flow Chart 74
C.2.1 Arduino GIGA Firmware Flowchart 75
C.2.2 PSC and EPICS Communication Flow 77
C.2.3 Software Configuration Management 77
C.3 Source Code 78

Chapter 1

Introduction

1.1 Project Overview

The Multi-Rail Isolated Power Supply Monitoring System is a hardware and software system developed to provide centralized monitoring of multiple electrically isolated power-supply rails. The system measures both output voltage and load current, provides configurable alarm limits, displays operating status and measurement data via a local display or ethernet for integration with an EPICS-based control system.

The completed system monitors six independent power-supply rails:

- +5VA
- +5VB
- +5V
- -5V
- +15V
- -15V

Because the monitored supplies may be electrically isolated from one another, the monitoring system must preserve that isolation while still allowing all six channels to communicate with a common controller. Each supply rail therefore uses an independent voltage/current monitoring channel and an isolated digital communication path.

The primary controller is an Arduino GIGA R1 WiFi. The controller acquires measurement data from the individual monitor channels, converts the data to engineering units, applies the appropriate rail polarity, evaluates configured alarm limits, and presents the resulting information on a GIGA Display Shield. An Ethernet interface provides network communication between the embedded controller and an EPICS Input/Output Controller (IOC). The EPICS IOC then makes the measurements, alarms, and system status available to a CS-Studio Phoebus operator interface.

1.2 Project Motivation

Power-supply systems containing multiple isolated voltage rails present a monitoring challenge because each circuit associated with one supply can't share a direct electrical reference with another supply. Connecting conventional measurement or communication circuits across these references can break isolation between the rails.

A centralized monitor is desirable because it allows the user to observe all supply voltages and currents from a single interface, identify abnormal operating conditions, detect communication or sensor faults, and integrate the power-supply status into a larger control system.

The design developed for this project addresses these requirements by performing voltage and high-side current measurement locally on each monitored rail and transferring the resulting information across electrically isolated digital interfaces. The controller receives measurement data from all six rails without requiring their power references to be directly connected together.

1.3 System Architecture

The monitoring system is divided into three primary functional areas: the power-supply monitoring hardware, the microcontroller, and the supervisory control interface.

The power supply monitoring hardware contains the current-sense resistors, LTC2945 PMICs, and digital isolation circuitry. Each LTC2945 measures the supply voltage and the differential voltage developed across a high-side current-sense resistor. The resulting digital measurement information is sent over the I^2C bus while maintaining the required electrical isolation.

The Arduino GIGA R1 WiFi provides the mcu processing functions. Its firmware initializes the monitoring channels, acquires voltage and current measurements, detects communication errors, converts raw measurements to engineering units, applies channel-specific polarity and scaling, evaluates alarm limits, and updates the local display. Configuration information, including network parameters and alarm limits, is retained in persistent storage so that required settings can survive a controller reset or power cycle.

Network communication is provided through an Ethernet interface using the PSC protocol. The EPICS IOC communicates with the embedded controller using PSCDriver and converts the transmitted values into EPICS process variables. CS-Studio Phoebus provides the remote operator interface for viewing measurements, alarm conditions, connection status, and system state.

1.4 Design Objectives

The major design objectives for the Multi-Rail Isolated Power Supply Monitoring System are to:

1. Measure the voltage of six independent power-supply rails.
2. Measure the load current of each rail using high-side current sensing.
3. Preserve electrical isolation between independently referenced power-supply rails and the common controller.
4. Convert acquired data into meaningful engineering units and correctly represent both positive and negative voltage rails.
5. Detect I^2C communication failures and prevent invalid sensor data from being presented as valid measurements.
6. Compare voltage and current measurements against configurable upper and lower alarm limits.
7. Provide a local graphical display of voltage, current, alarm status, communication status, and network state.
8. Retain configuration and alarm settings through controller resets and power cycles.
9. Provide Ethernet communication for remote monitoring and configuration.
10. Integrate the monitoring system with an EPICS IOC and CS-Studio Phoebus operator interface.
11. Maintain the hardware, firmware, and control-system software under revision control so that changes made during design, implementation, and testing remain traceable.

1.5 Project Scope

The scope of the project includes the design and implementation of the sensor interface hardware, Arduino GIGA controller hardware, embedded firmware, Ethernet communication interface, EPICS IOC, and operator-interface software. The hardware portion includes the six voltage/current monitoring channels, current-sense resistors, digital isolation circuitry, controller interface, Ethernet circuitry, and supporting power-supply circuitry.

The software portion includes the embedded acquisition and alarm-processing firmware, persistent configuration storage, local display interface, PSC network communication, EPICS IOC configuration and database records, and the CS-Studio Phoebus operator interface.

Design files and software associated with the project are maintained in the project Git repository:

`https://github.com/capotosto/PS_Monitor`

The repository provides revision control for the hardware design, firmware, EPICS IOC files, documentation, and other project artifacts generated during development.

1.6 Report Organization

This report documents the development of the monitoring system from preliminary design through implementation and testing. The preliminary-design chapter defines the major functional blocks, presents the system architecture, documents supporting engineering calculations, includes the hardware schematics and bills of materials, and describes the primary firmware and control-system software flows.

Subsequent chapters document prototyping and development activities, the EPICS IOC implementation, and the CS-Studio Phoebus operator interface. Supporting project-management and design documents developed during the earlier phases of the capstone project are retained as part of the overall project documentation and provide traceability between the original requirements, design decisions, implementation, and final system.

Chapter 2

Problem Background and Definition

2.1 Purpose and Scope

This chapter documents the hardware and software design of the Multi-Rail Isolated Power Supply Monitoring System. The design monitors six power-supply rails, measures rail voltage and load current, preserves electrical isolation between floating supplies and the controller, evaluates measurements against user-configurable alarm limits, presents local status on the controller display, and publishes measurements and status to an EPICS control-system interface.

This design is intended to establish the major analog and digital functional blocks, document supporting engineering calculations, define the hardware schematic baseline, and provide software flowcharts detailed enough to support implementation.

2.2 Problem Definition

The project addresses the need to monitor several low-voltage power-supply rails that do not necessarily share a common reference. A conventional monitoring circuit that directly connects the grounds together would defeat the electrical isolation of the supplies and could create unwanted return paths. However, the user needs a single location from which to observe rail voltage, rail current, alarm state, communication status, and the overall condition of the power system.

The required monitoring system must therefore make accurate measurements on six separate rails while preserving isolation between the monitored domains. It must support both positive and negative rails, detect sensor or communication faults, provide configurable voltage and current limits, retain configuration information after reset, present useful local status, and make the same data available to the facility control system over Ethernet. The monitored rails in the present design are +5 V A, +5 V B, +/-5 V, and +/-15 V.

2.3 Proposed Solution

The proposed solution assigns one LTC2945 voltage/current monitor to each power rail. Load current is converted to a small differential voltage by a precision high-side shunt resistor and the LTC2945 digitizes both the shunt voltage and the rail voltage. Each sensor remains referenced to its own monitored supply domain. The I^2C communication path is then isolated so that digital measurement data can be transferred to a common Arduino GIGA R1 WiFi controller without directly bonding the isolated supply returns together.

The Arduino GIGA performs channel acquisition, engineering-unit conversion, negative-rail sign handling, alarm evaluation, configuration storage, display updates, and network communication. A local GIGA Display Shield provides immediate status at the hardware. Ethernet transports measurement and status data using the PSC protocol to an EPICS IOC, where PSCDriver and EPICS records make the values available to a CS-Studio Phoebus operator interface. This architecture separates the rail-side analog measurement functions from the controller and supervisory-control functions while maintaining a single user interface for the complete system.

Chapter 3

Technology Solution

Chapter 4

Chosen Conceptual Design

The major system functions are summarized below.

1. **Power-Supply Interface:** Connect to the six monitored rails and route each rail through the associated voltage/current monitoring channel.
2. **Current Measurement:** Convert load current to a differential voltage across a precision shunt resistor and digitize it with the LTC2945.
3. **Voltage Measurement:** Measure the monitored rail voltage using the LTC2945 supply-voltage ADC channel.
4. **Electrical Isolation:** Maintain galvanic isolation between each floating monitored supply and the common controller-side I^2C bus.
5. **Digital Acquisition:** Poll each monitor, identify communication faults, apply channel-specific sign/scaling information, and calculate engineering-unit values.
6. **Alarm Processing:** Compare voltage and current values against configured upper and lower limits and generate channel alarm states.
7. **Local User Interface:** Present all rail measurements, alarms, network state, and communication status on the Arduino GIGA Display.
8. **Network/EPICS Interface:** Publish measurements, alarm limits, alarm state, and connection status over Ethernet for the EPICS IOC and Phoebus interface.

4.0.1 Functional Block Diagram

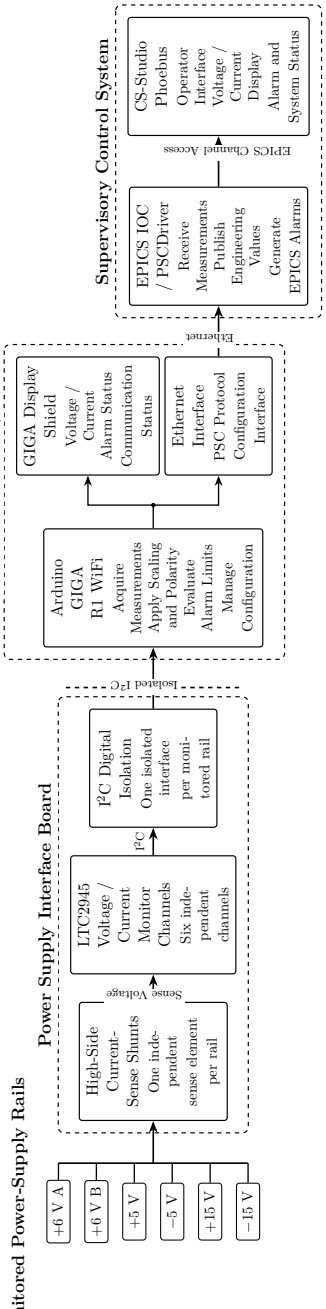


Figure 4.1: Functional block diagram of the multi-rail isolated power supply monitoring system. Six independent power-supply rails are measured using high-side current sensing and LTC2945 voltage/current monitoring channels. Digital isolation preserves electrical independence between the monitored supplies and the common controller. The Arduino GIGA processes the measurements, provides a local operator display, and transmits measurement and status information over Ethernet to the EPICS IOC and CS-Studio Phoebus operator interface.

Chapter 5

Potential Social, Ethical, Legal, and Environmental Impacts of the Project

5.1 Societal Impact

The principal societal impact of the monitoring system is the improved data availability. The six isolated power supplies that would otherwise have to be monitored individually can now be measured from one local display and from the EPICS control system. Abnormal voltage, current, or communication conditions can be detected faster. In a laboratory or industrial setting, this can reduce troubleshooting time and help to avoid operation of the equipment with a power supply failure.

There is also a obligation associated with presenting measurements to an operator. Erroneous data can be more problematic than unavailable data if an operator believes that a failed measurement is a good one. For this reason, communication failure detection was included in the design. The firmware distinguishes an failed or offline sensor from a legitimate zero measurement, and presents the communication state together with the measured values. This was an important design decision since the monitor is meant to present information which an operator can rely on for making maintenance and operational decisions.

5.2 Global Impact

The design is dependent on a global electronics supply chain. ICs, passive components, PCB fabrication, and commercial Arduino modules can be manufactured in several different countries even when they are supplied through United States distributors. This gives access to a wide range of components and lower manufacturing costs, but at the same time creates risks associated with component availability, obsolescence, shipping disruptions, and supplier dependence.

The design mitigates some of these risks through the use of standard component packages, conventional PCB assembly, repetitive circuit blocks, and standard interfaces such as I^2C

and Ethernet. makes qualification of replacement components or alternate manufacturing sources much easier than it would be with highly proprietary hardware.

Global sourcing creates an ethical obligation that is not completely covered by component price and technical performance. For a production version of the monitor, the selection of a supplier should also include consideration of responsible labor practices, material sourcing, regulatory compliance, and traceability of the supplier. These considerations considerations would become increasingly important for large scale production of the design.

5.3 Legal and Regulatory Impact

This project is an engineering prototype and can't be considered to have any system-level regulatory approval solely because individual components carry approvals or compliance markings. If the monitor were converted into a commercial product, the required regulatory evaluation would depend on the installation, enclosure, power source, network interfaces, and markets in which it would be sold.

For example, no attention was paid to RoHS compliance, as the boards are intentionally assembled with leaded solder, so leaded HASL or ENIG were both acceptable ordering options. Use of the monitor with leaded solder however, would be an issue for many markets with RoHS mandates.

EMC, product-safety, and radio-frequency requirements would also have to be identified for the final installation before the production product could be offered for sale. EMC lab compliance testing can be extremely expensive.

The distinction between component compliance and system compliance is therefore important. The design can take advantage of the recognized components and accepted engineering practice without implying any approval not obtained for the finished assembly. This can most easily be related to the idea of "UL Listed" and "UL Registered" (UL vs UR labels) - UL listed components are certified on their own standing, but UR components need to be certified in the installation, and are only a recognized component of a system.

5.4 Ethical Impact

The first ethical obligation associated with the monitor is the accurate and unambiguous presentation of the measured information. The system should not hide sensor failures or substitute erroneous data with apparently valid measurements. The implemented communication failure detection, alarm indication, and separate sensor failure reporting were included partly for this reason.

The next consideration is network security. The current HTTP and PSC interfaces are designed for use in a controlled local network. They do not have security features necessary to be connected to an untrusted network. If the design were deployed in an unprotected network, additional requirements for authentication, access control, encryption, network seg-

mentation, and configuration management should be dfined. Any possibility of unauthorized modification of alarm limits or the network configuration would undermine the reliability of the monitoring system, even if the measurement hardware would continue operating properly.

Ethical considerations also also relevant to procurement. Cost and availability are important engineering considerations, but they should not be the only criteria for selecting a production supplier. Labor practices, material sourcing, counterfeit parts risk, and the ability to provide traceable parts are also appropriate considerations for a product moving from prototype quantities into production.

5.5 Environmental Impact

The main environmental impacts of the monitor come from PCB fabrication, semiconductor manufacturing, raw-materials, transportation, electricity consumption, and eventual electronic waste. It's impossible to make a design with zero environmental impact, but some aspects of this design reduce the quantity of hardware that would need to be disposed of during the useful service life of the product.

The Sensor Board, Arduino Shield, Arduino GIGA, display, and other major assemblies are separable. Therefore, a failed subsystem can therefore be repaired or replaced independent of the complete monitoring system. Standard component packages and commercially available modules were preferred where possible, and this increases the probability of replacement of an individual component. Alarm limits and network configuration are stored in software which allows changing the configuration without producing a new PCB.

For production quantities, RoHS-compliant components should be used where possible. The end-of-life plan should include recovery or recycling of the electronic assemblies. These considerations align with RoHS and WEEE requirements to encourage collection, reuse, recovery, and recycling of electronic equipment.

5.6 Relationship to the Original Requirements

Several of the societal and ethical considerations described above were indirectly represented in the original requirements definition. Electrical isolation was explicitly required to allow the monitoring of one rail would not defeat the isolation of the other. The communication fault detection was required to ensure erroneous data would not be interpreted as a valid electrical measurement. User-configurable alarm limits, local status indication, EPICS monitoring, and persistent configuration storage were also included as functional requirements. These requirements increase the reliability, maintainability, and the quality of the information presented to the operator.

Environmental considerations were introduced later in the design through serviceability, component reuse, and the use of common parts. Regulatory certification, ethical supplier selection, and cybersecurity requirements were not among major items in the original re-

quirements definition. If the prototype were developed into a production product, these aspects should be transformed into explicit requirements with acceptance criteria.

Chapter 6

Design Methodology

Chapter 7

Simulation Results

Simulations were performed using LTSpice in a DC Operating Point simulation to verify the calculation for the full-load shunt voltage for each of the voltages. Only the positive polarity was tested in the simulation, as negative polarity will naturally not matter to a a passive element such as the resistor.
The data collected in this simulation matches the data found in Appendix A section B.1, table B.1

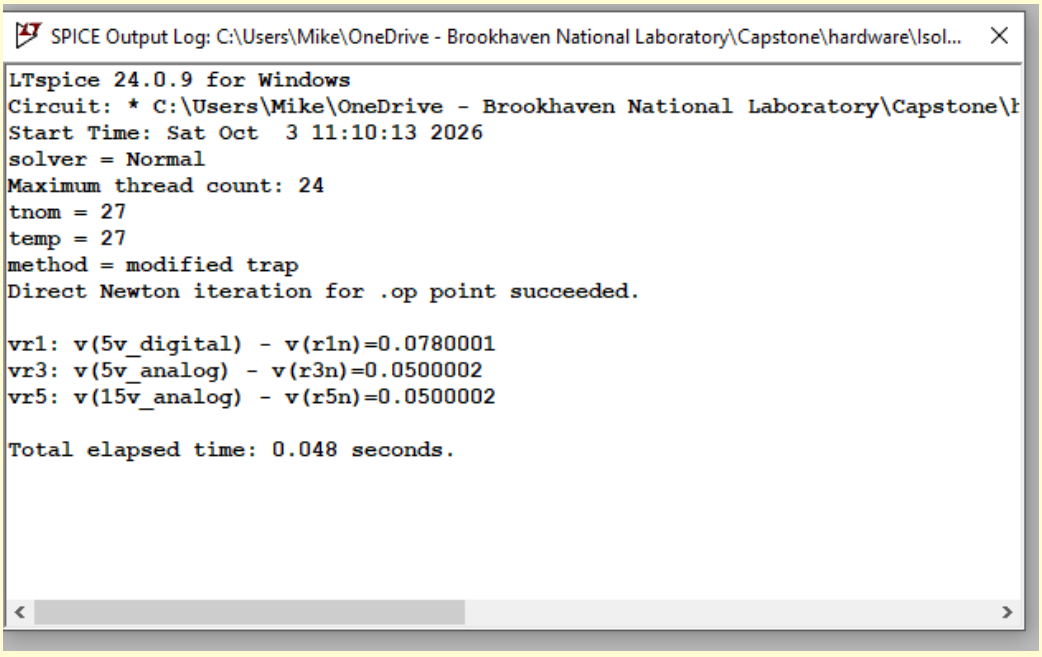


Figure 7.1: Simulation verifying current shunt voltage drops, data log

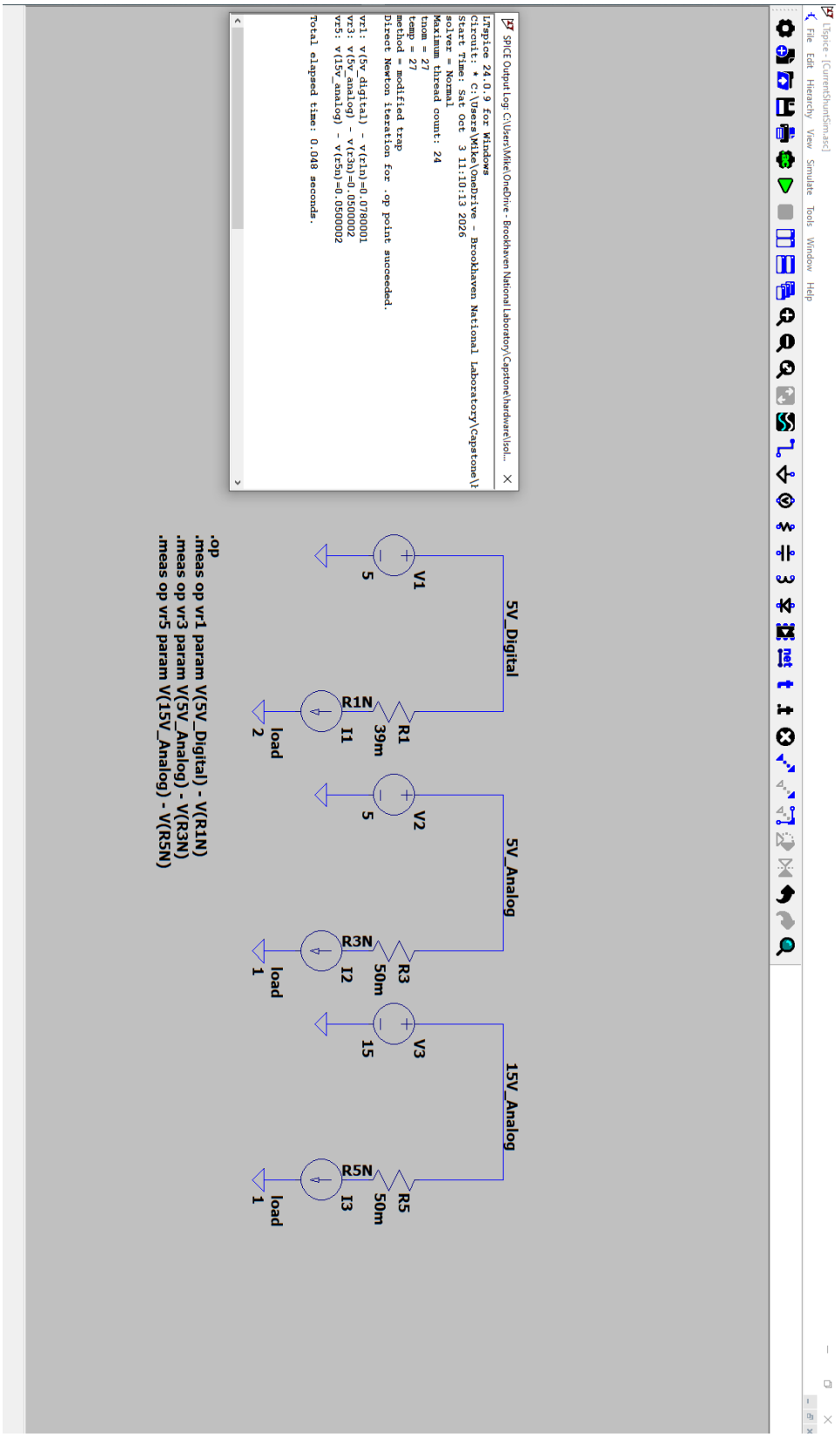


Figure 7.2: Simulation verifying current shunt voltage drops, full simulation screenshot

Chapter 8

Final Design

Chapter 9

Solution Implementation

9.1 Testing the LTC2945

The LTC2945 testing was carried out using a Linear Tech eval board, DEMO CIRCUIT 1697A. Connections and modifications were made as follows:

- VIN: tied to Arduino +5V
- GND: tied to Arduino GND
- ADIN: Tied to GND
- SCL: Tied to Arduino Pin D8 I2C2 SCL
- SDAI/SDAO (JP2 set to TIE): Arduino Pin D9 I2C2 SDA
- VPU: Tied to Arduino +3v3
- VOUT: Float or tied to dummy load (1kΩ to 10kΩ)
- 20mΩ shunt replaced with 17Ω shunt for testing with ∼5kΩ load
- Installed 4.7kΩ pull up on I2C lines via R4, R5 footprints on EVB

9.1.1 Calculating Shunt Value for Test

Using the 5V from the arduino as a source, and a load ranging 1k to 10k ohms, we calculate:

$$R_{L,min} = 1k\Omega$$

$$V_{shunt} = V_S \frac{R_S}{R_L + R_S}$$

$$R_S = \frac{V_{Shunt} R_L}{V_s - V_{Shunt}}$$

$$V_{shunt} = 0.1024V$$

$$R_{S,fullscale} = \frac{(0.1024V)(1k\Omega)}{5V - 0.1024V} = 20.91\Omega$$

Adding some headroom, say, 15%, and going to an E96 resistor value...

$$R_S = 0.85(20.91) = 17.8\Omega$$

Verifying for the high end of the range:

$$I_{min} = \frac{5V}{10k\Omega + 17.8\Omega} = 499.1\mu A$$

$$V_{shunt,min} = (499.1\mu A)(17.8\Omega) = 8.88mV$$

The shunt will range from 8.88mV to 87.44mV, within the ADC's usable range of 0V to 102.4mV.

9.1.2 Pictures of Evaluation Board, Display, and Phoebus Panels

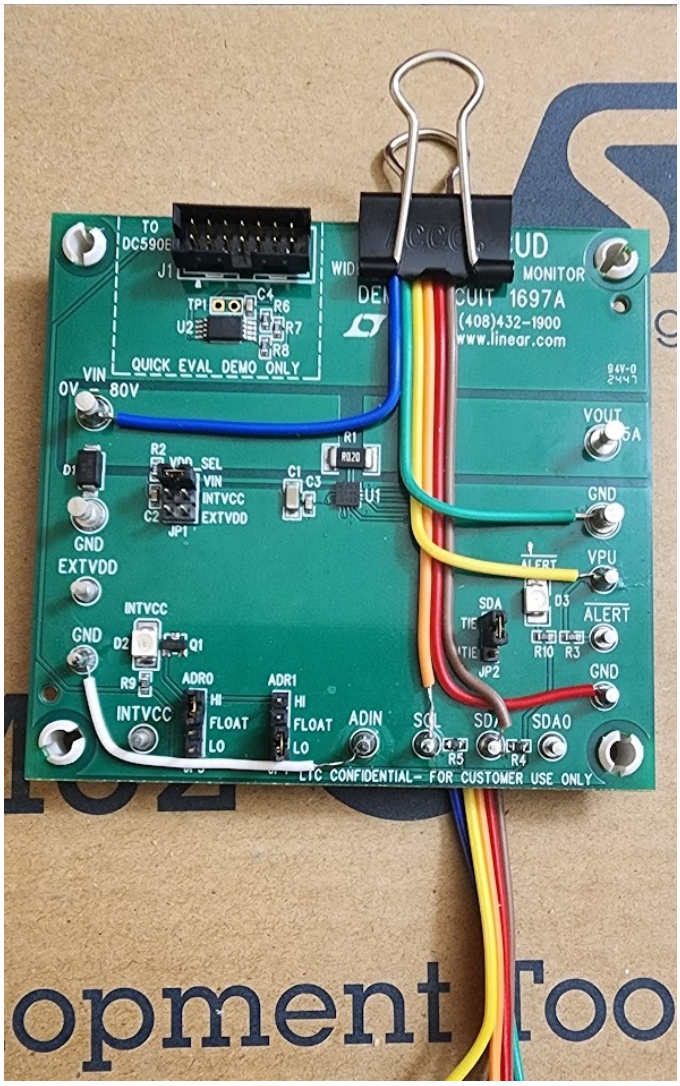


Figure 9.1: DEMO CIRCUIT 1697A Eval Board with Original Shunt, fly leads to connect to Arduino.



Figure 9.2: DEMO CIRCUIT 1697A Eval Board with 17Ω Shunt, fly leads to connect to Arduino, and 4.7kΩ test load.

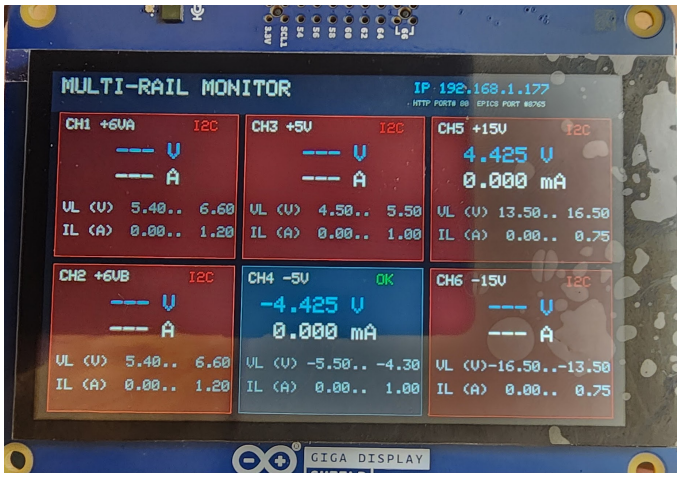


Figure 9.3: Arduino Giga with GIGA DISPLAY SHIELD showing dummy -5V rail being monitored, without alarm

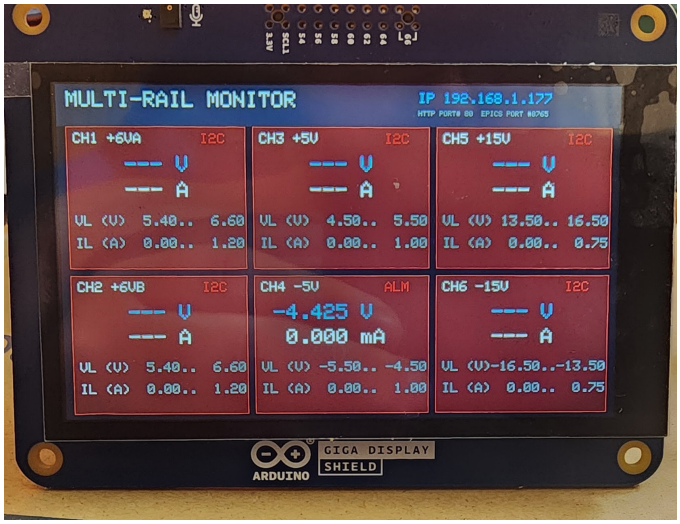


Figure 9.4: Arduino Giga with GIGA DISPLAY SHIELD showing dummy -5V rail being monitored, with UVL alarm, other channels I2C ALARM.

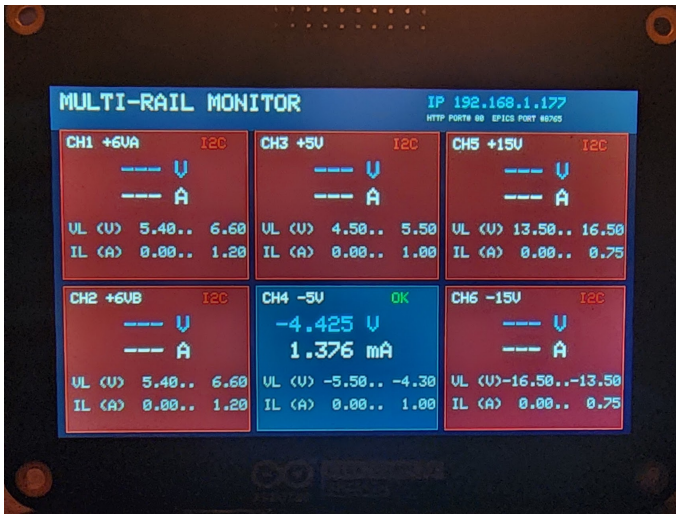


Figure 9.5: Arduino Giga with GIGA DISPLAY SHIELD showing dummy -5V rail being monitored, without alarm, 3.3kΩ Load



Figure 9.6: Arduino Giga with GIGA DISPLAY SHIELD showing dummy -5V rail being monitored, without alarm, 3.3kΩ Load



Figure 9.7: Arduino Giga with GIGA DISPLAY SHIELD connected to an LTC2945 eval board

9.2 Bring Up and Testing of the First Spin Board

After construction of the printed circuit boards was complete, the board bring-up process was carried out prior to system-level testing. The objective of the board bring-up process was to ensure that the electronic operation of the circuitry was functioning, locate assembly

problems and communication issues, and make modifications if required before running the entire system.

In this case, initial testing was not done by powering up and trying to run the entire system, but was carried out incrementally. Each rail, each signal and functional block was tested separately so as to pinpoint failures without adding other variables. Measurements were made with the use of a digital multimeter and oscilloscope, whereas firmware and serial diagnostic output was used to ascertain if the Arduino was communicating with peripheral devices.

Several problems were noted during this phase, but of primary concern was an error in the I^2C Serial Data (SDA) interface between the circuitry of the Arduino side and the isolated circuitry. This problem led to a failure of I^2C communication and troubleshooting had to be done at the signal level with an oscilloscope. The troubleshooting revealed that there was an incorrect selection of a component in the constructed board, and temporary modification of the board was done with the help of an additional logic inverter to fix the problem and allow further development and testing of the system.

The subsequent sections describe the board bring-up process, detected failures, measurements made for diagnostics, performed modifications, and the testing of the fixed board.

9.3 Assembly Verification/Visual Inspection and Smoke Test

9.3.1 Visual Inspection

I hand-assembled all components of the boards, so a thorough visual inspection was carried out. The board was assembled using water-soluble paste flux and organic core water soluble 63/37 Sn/Pb solder wire (0.031" and 0.015"). As such, the boards were washed thoroughly prior to inspection.

Only one notable finding was found, U20 was shifted off the pads, with the exposed pad in the center having 'snapped' to pins 4/5/6. The board was reworked via hot air prior to power being applied to correct this. No other defects were found on visual inspection.

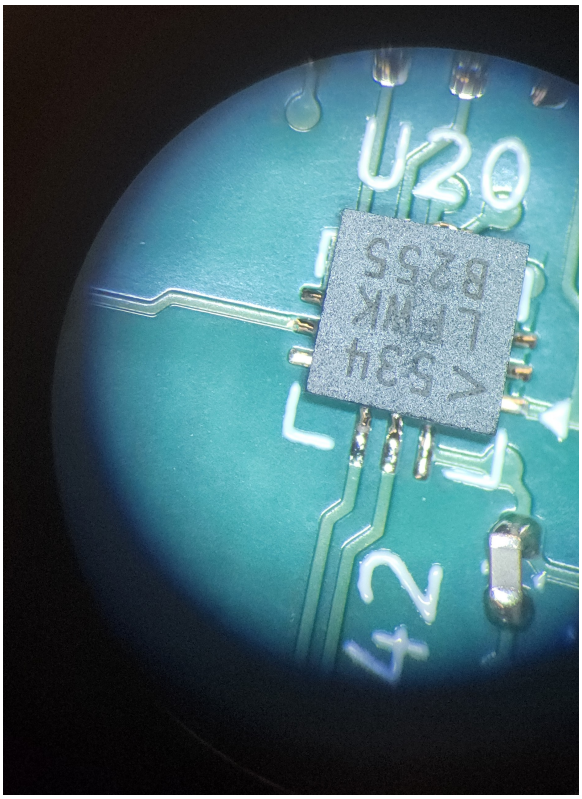


Figure 9.8: The PMIC, U20, was found to have not been properly aligned when soldered initially, requiring rework to re-center the component.

9.3.2 Smoke Test

An open and short test was performed on the input and output side of the 'Sensor' board's regulator. Both the input and output side showed a charging capacitive load on the DMM (increasing resistance), and so it was deemed safe to proceed to power on testing.

A +5V supply was connected to the input, set for a 100mA current limit, and the board came up to approximately 3.6V. The current limit was increased to 500mA, and the bench supply came up to 5V and regulated. The output of the on-board regulator was verified to be putting out 3.3V. The microcontroller was observed to boot properly.

9.4 I^{2}C SDA Communication Problem

9.4.1 Initial Symptoms

One of the first significant problems encountered during board bring-up was a failure of the isolated I^{2}C interface. The Arduino firmware attempted to communicate with the monitoring devices, but the expected responses were not received. Because I^{2}C communication requires

both the Serial Clock (SCL) and Serial Data (SDA) signals to operate correctly, the two signals were examined at multiple locations on the PCB using an oscilloscope.

During the initial measurements, unusual waveform shapes were observed on portions of the isolated interface. These measurements were initially misleading because the oscilloscope reference and the isolated circuit grounds were not connected as intended. As I was taking measurements, working only on the +6VA supply initially to focus testing, I was measuring on both isolated and non-isolated side of the bus, and I was using floating bench supplies. After connecting all the supply commons, the waveforms were reasonable.

The SCL signal was then observed switching as expected, demonstrating that the Arduino was attempting to generate I^2C transactions. The SDA path, however, did not behave correctly. At one point in the signal chain the clock activity was clearly visible, while the corresponding SDA-side output remained at a nearly constant logic level. This indicated that the failure was not caused by the Arduino failing to initiate communication, but was instead located within the SDA interface circuitry.

With the sensor board disconnected, I was able to observe clock and data transmissions, with NACK visible on the 9th clock as below:

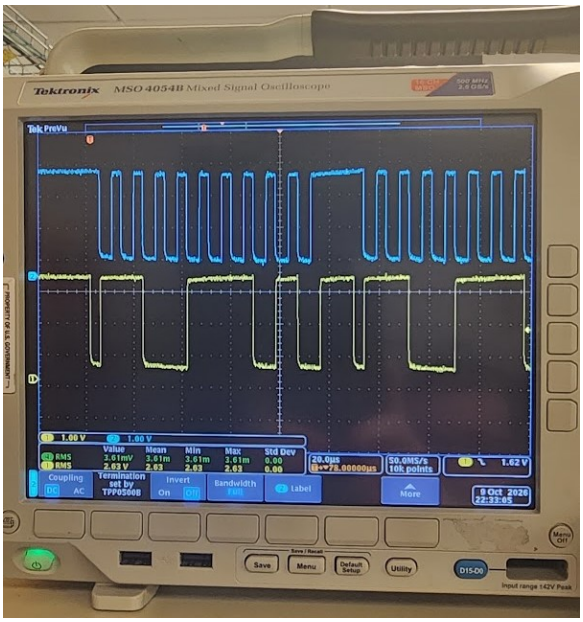


Figure 9.9: I2C messages on Arduino shield cable driver output, with sensor board disconnected, NACK bit on 9th clock pulse

9.4.2 SDA Signal Troubleshooting

Troubleshooting proceeded by following the SDA signal through the circuit one node at a time. The signal was measured on both sides of the isolation circuitry and at the input and output of the associated logic circuitry. This approach made it possible to determine where the expected transitions stopped propagating.

Testing showed that the input side of the isolation circuit was responding to SDA activity. The voltage measured across the input device during a low state was approximately 1.34 V, indicating that the input element was being driven. However, the corresponding output did not reproduce a valid SDA waveform.

Additional testing was performed with the pull-up and drive circuitry connected and disconnected. The current limiting resistor for the optoisolators was dropped from 2k2Ω to 909 Ω resistor to ensure the drive current wasn't marginal, and the observed nodes only fell to approximately 1.8 V during what should have been a logic-low condition. This level was too high to provide a reliable digital low for the following logic stage. It was behaving almost as if two push-pull outputs were fighting each other...

9.4.3 Identification of Incorrect Component

At this stage, I focused my attention around the PMIC's $\overline{SDAO}$ signal...Measuring the input pin of the optocoupler, I observed the line was being held high! This was expected behavior for I2C, except the selected part was designed to have an inverted SDA output!

Review of the schematic, assembled board, and component ordering information revealed that the part populated in the SDA interface was not the intended device configuration. Although the installed part was electrically similar to the device considered during the design, its logic behavior did not match the requirements of the implemented circuit.

I had ordered from Digikey the LTC2945IUD....the part required was the **LTC2945IUD-1**.

As a result, the output signal had the wrong logical relationship to the SDA signal required by the remainder of the interface. The PCB therefore could not be corrected through firmware alone. The hardware signal needed to be inverted before being applied to the following stage.

This problem was not immediately obvious during assembly because the component was physically compatible with the PCB footprint.

9.4.4 Inverter Bodge Modification

To continue board development without waiting on a parts order, a hardware bodge was implemented to correct the SDA signal polarity. A Texas Instruments SN74LVC2G04DCKR dual inverter was used for the modification, as an inverter I had on hand in the shop that was 3.3V compatible. The device contains two independent logic inverters and operates from single supply voltage range compatible with the logic used on the board. The DCK package is an SC-70-6 which is easily installed to the board with epoxy and tied into the circuit with 32awg magnet wire.

For the inverter channel used in the modification, pin 1 is the logic input and pin 6 is the corresponding inverted output. The affected SDA-related signal was therefore rerouted through this inverter before being connected to the following stage. The modification provided the additional inversion required to produce the logic polarity originally expected by the circuit.

The modification can be represented conceptually as

$$V_{\mathrm{OUT}} = \overline{V_{\mathrm{IN}}}, \tag{9.1}$$

where V_{IN} is the signal produced by the existing interface and V_{OUT} is the corrected signal applied to the following FET.

The bodge required cutting or and lifting the trace between the iso and the fet's gate, and adding short jumper wires between the existing circuit, the inverter input, and the fet gate. Power and ground were also provided to the inverter by tying into the optoisolator's 3.3V digital supply (via it's Pin 8, and the ground via pin 4).

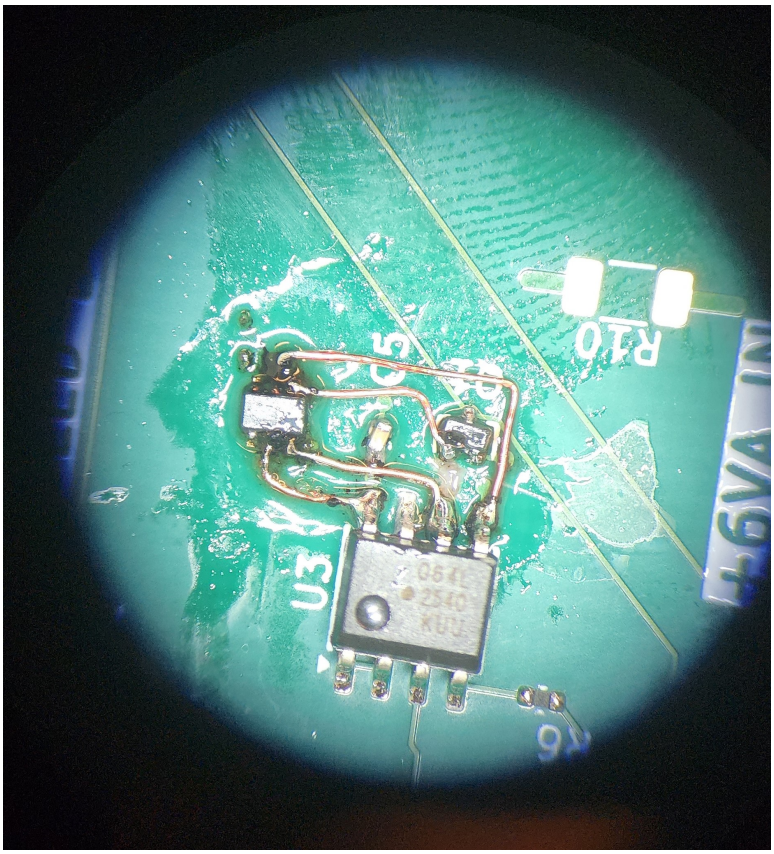


Figure 9.10: SC70-6 package inverter epoxied to the board, and 32awg bodge wires added, with cut trace, between optocoupler output and FET gate

9.4.5 Verification After Modification

After implementing the SDA modification, the SDA signal was inspected once again using an oscilloscope at several points in the communication circuit. The reason for this inspection was not only to check the presence of a waveform, but also to see whether there was proper propagation of logic-high and logic-low signals through the entire interface.

This signal was checked alongside the SCL waveform during the I^{2}C communication by the Arduino. The focus of observation was on the ninth clock pulse, which is where an I^{2}C slave should acknowledge the transmission by lowering the SDA signal. This observation was important since it showed that the process was moving further than before modification.

After confirming the waveforms, once again, communication was tested through the firmware. Success in communication after modification proved that the error occurred at the hardware stage of the SDA signal implementation and not within the I^{2}C software itself.

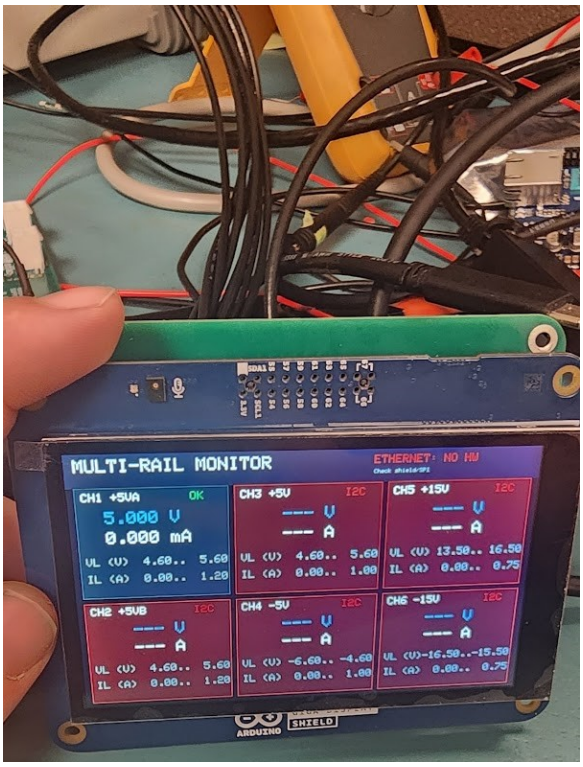


Figure 9.11: Communication is now working for the +5VA channel that has had the I2C connection repaired. The other 5 channels have had the FET gate floated (copper traces cut and lifted) from the opto to release the data bus, but have not yet had their inverters installed.

The troubleshooting process used to locate the problem can be summarized as:

1. Confirm that the required circuit power supplies are present.
2. Establish the correct oscilloscope ground reference for each isolated portion of the circuit.
3. Verify that the Arduino generates I^{2}C clock activity.
4. Probe SDA sequentially through each stage of the interface.
5. Identify the point where valid SDA transitions are lost.

6. Compare the installed component and its logic behavior with the schematic requirements.
7. Modify the circuit to correct the signal polarity.
8. Re-measure the SDA waveform.
9. Repeat the firmware communication test.

The error was helpful in the bring-up of the board since it showed that embedded system debugging must consider both the hardware and the software at the same time. The error of the firmware showing communication problems via I^2C showed the symptom but using the oscilloscope to measure the values pointed out where the exact problem was located.

9.4.6 Further Bodges and Repairs

Repairs were carried out on the remaining 5 channels. Additionally, it was found that the +5VA and -5V both had the same I2C address configured; this was modified by removing U20, cutting the traces to the address pins, and running 30awg magnet wire to tie Pin 3 to INTVCC (via U21-8) and Pin 2 to -5V via C43-2. Power was restored and then it was observed to now be working on all 6 channels!

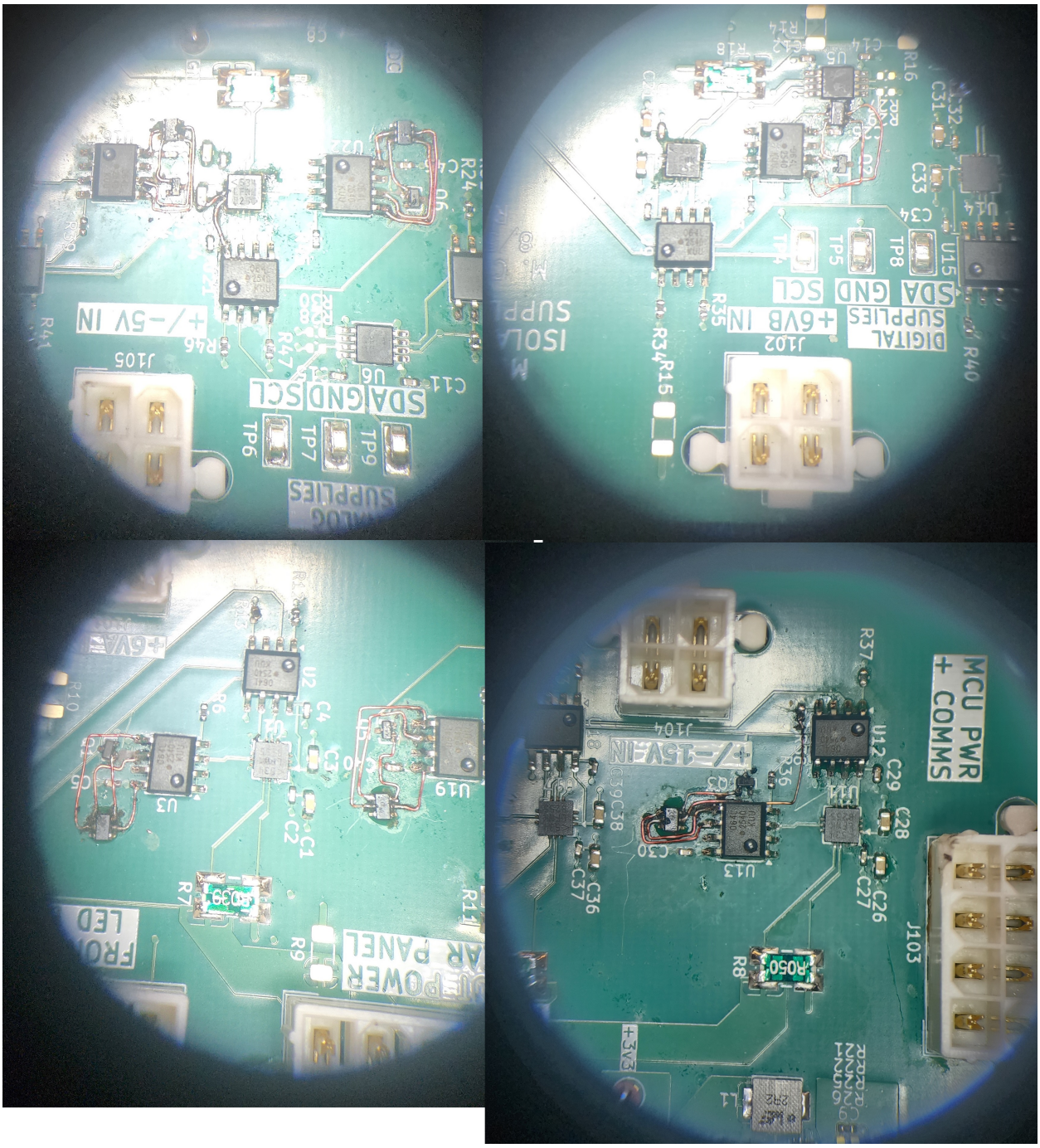


Figure 9.12: Rework is completed for all 6 channels

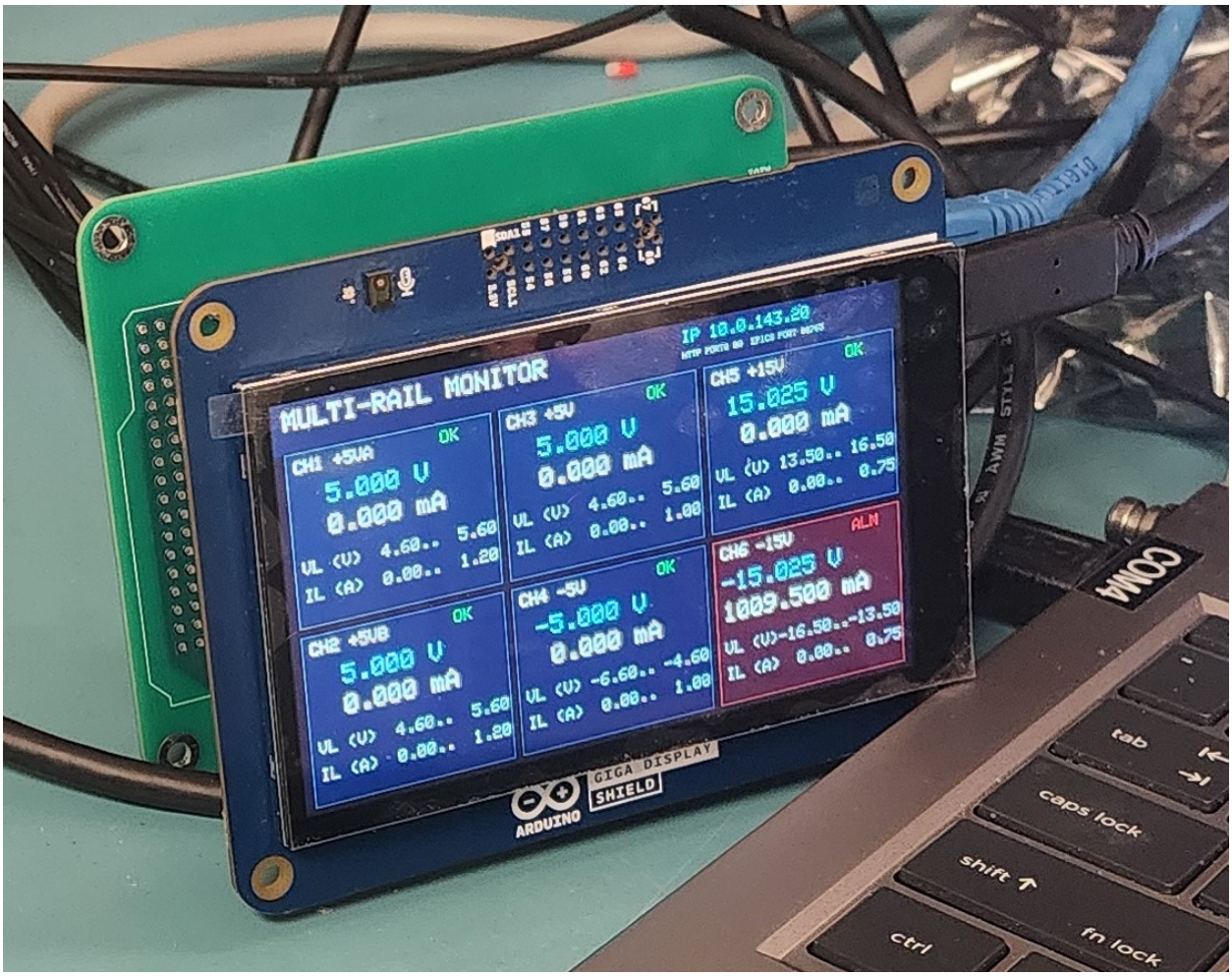


Figure 9.13: All 6 channels now working!

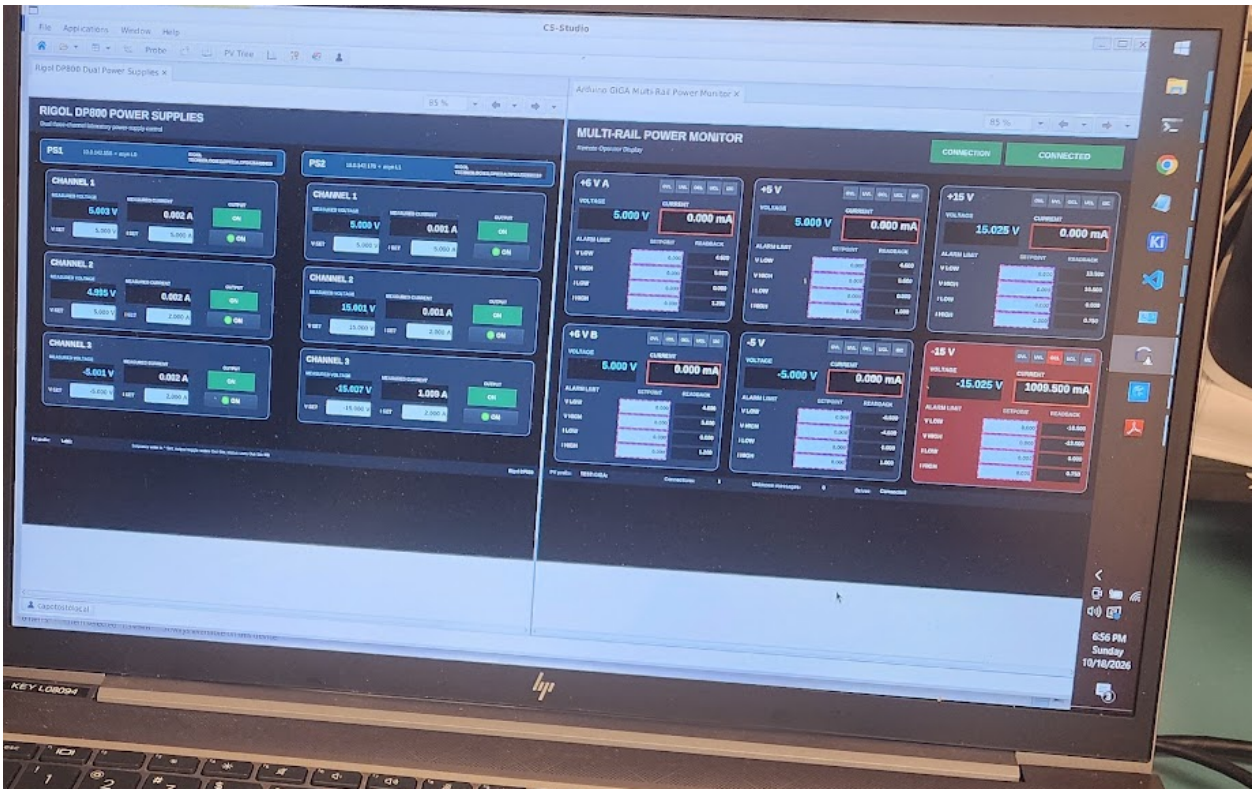


Figure 9.14: Control Panels working as well, showing 1A dummy load on one channel

Chapter 10

Evaluation of Solution's Efficacy

10.1 Automated Functional Tests

10.1.1 System Under Test

The system under test is the completed six-channel power supply monitoring system developed for this capstone project. The system monitors the following power rails:

- $+5\,\mathrm{V}A$
- $+5\,\mathrm{V}B$
- $+5\,\mathrm{V}$
- $-5\,\mathrm{V}$
- $+15\,\mathrm{V}$
- $-15\,\mathrm{V}$

Each rail has its own separate voltage and current monitoring. The measurement circuits are isolated from the communication channel on the Arduino side and are connected to Arduino GIGA R1 WiFi. The Arduino has a local graphical display by using GIGA Display Shield and Ethernet connection by a custom interface shield I've built. The Ethernet connection works with EPICS Input/Output Controller (IOC), making it possible to monitor the rails measurement and status remotely.

The testing of the system checks out the entire system but not each circuit block separately. This test involves the sensor circuits, isolation circuits, Arduino firmware, display, Ethernet connection, EPICS IOC, and related software as one whole system.

10.1.1.1 Test Objectives

The primary objectives of the system test are to verify that:

1. the complete system powers on and initializes without errors;
2. all six power rails are identified and displayed correctly;
3. the voltage measurement for each rail agrees with an external reference measurement within the specified system tolerance;
4. the current measurement for each rail agrees with the applied load within the specified system tolerance;
5. positive and negative supply rails are reported with the correct voltage polarity and current convention;
6. measurements shown on the local display correspond with the values available through EPICS;
7. voltage and current operating limits produce the appropriate status indication;
8. user-configurable settings remain stored following a power cycle;
9. Ethernet or IOC communication can be interrupted and restored without requiring a complete reprogramming of the monitor;
10. measurement data can be recorded and used to generate a permanent test report; and
11. the completed system can successfully perform the functions required for the final capstone demonstration.

10.1.2 Scope of Testing

10.1.2.1 Features to be Tested

System testing will include the following functions:

- system startup and initialization;
- detection and monitoring of all six power rails;
- voltage measurement;
- current measurement;
- measurement response to changes in load;
- correct handling of positive and negative rails;

- local GIGA Display operation;
- Ethernet communication;
- EPICS process-variable updates;
- voltage and current limit/status indications;
- storage and restoration of persistent settings;
- communication-loss recovery;
- repeated measurements to identify intermittent operation; and
- automatic recording and reporting of system test results.

10.1.2.2 Features Not Included in This Test

The system test is intended to verify the functional requirements of the prototype only. The following tests are therefore outside the scope of this system-level test plan:

- EMC testing;
- qualification tests for environmental temperature/humidity;
- qualification tests for vibrations or mechanical shock;
- qualification tests for life cycle/accelerated aging;
- destructive overvoltage tests; and
- destructive overcurrent tests.

10.1.3 Test Equipment and Software

The following equipment and software will be used during system testing:

Equipment/Software	Purpose
Power supply monitor prototype	Complete hardware and firmware system under test.
DC bench supplies	Provide the six input rails required by the monitor.
Rigol DL3021 electronic load	Provide programmable and repeatable loading of individual power rails.
Computer running EPICS IOC	Receive voltage, current, and status information from the Arduino over Ethernet.

Equipment/Software	Purpose
Phoebus/EPICS display	Provide operator-level observation of process variables and system status.
Python scripts	Control the electronic load, acquire EPICS values using PyEPICS, save raw test data, and automate portions of the test sequence.
Ethernet network	Provide communication between the Arduino and EPICS IOC.

10.1.4 Test Configuration

During testing, the assembled power supply monitor will be operated in its normal system configuration. All six monitored supplies will be connected to the appropriate monitoring inputs. The Arduino, GIGA Display Shield, custom monitoring hardware, and Ethernet interface will be powered and operating simultaneously.

For measurements involving the programmable electronic load, only the rail under test will be connected to the electronic load. The remaining rails will remain energized so that the monitor continues operating as a complete system, and may or may not be connected across resistive loads.

The electronic load measurement will provide the reference for both load voltage and current.

10.1.5 System Acceptance Criteria

The test is intended to establish whether the assembled prototype satisfies its functional requirements. The following system-level acceptance criteria will be used.

Parameter	Acceptance Criterion
System startup	System initialization should not involve a firmware error or user action, and all six rails should be shown.
Voltage measurement	The monitor reading should match the external reference voltage within ±2% across the full tested operating range.
Current measurement	Monitor reading should match the reference current within ±5% across the full intended operating range. In addition, the uncertainty and resolution of the reference device should be considered at low currents.
Positive/negative rail determination	Each measurement should use the correct polarity. The current in the negative channel should follow the sign convention used in the firmware.

Parameter	Acceptance Criterion
Display functionality	The display should show the proper identification of the rail, along with the rail voltage, current, and limits.
EPICS operation	All process variables should communicate and report their status without errors in normal operations.
Display and EPICS agreement	Local and remote readings should be identical to the same underlying measurement. Differences from rounding to integer in display are allowable.
Limit status	Any programmed limit should change status as the corresponding measurement crosses the limit and return to the normal status upon the measurement returning to the allowable range.
Persistent configuration	Any stored settings should be preserved after powering down and up the system.
Network recovery	After the restoration of Ethernet communication, EPICS should continue to update without any firmware reprogramming.
Test data	Each automatic test should yield a recorded raw-data file allowing for test results reconstruction.

The voltage and current accuracy requirements above are system-level requirements rather than component-level accuracy specifications. They include the combined effects of the sensor, shunt resistor, analog-to-digital conversion, calibration, wiring, firmware calculations, and reference-instrument uncertainty.

10.1.6 System Test Matrix

The system tests that are planned are listed in Table 10.3 below.

Table 10.3: System Level Test Matrix

ID	Test	Method	Pass Criterion	Result
ST-01	Power On and Initialization	Apply power to the entire system and check the operation of Arduino, the local display, the sensor channels and the network interface.	The system initializes properly, all six channels are operational.	☐ Pass ☐ Fail ☐ Retest
ST-02	Rail Identification	Apply power to the six rails and correlate the displayed channel with the corresponding input physical connection.	The correct rail identification with the right polarity occurs.	☐ Pass ☐ Fail ☐ Retest
ST-03	Voltage Accuracy	Measure each rail using a DMM or the E-Load and compare the reference value with the local and EPICS values at different operating points.	The error is within the specified voltage tolerance.	☐ Pass ☐ Fail ☐ Retest
ST-04	Current Accuracy	Apply different loads to each rail using the programmable electronic load and compare the reference current with the monitor measurement.	The error is within the specified current tolerance.	☐ Pass ☐ Fail ☐ Retest
ST-05	Dynamic Load Response	Change the current of the electronic load while monitoring the local display and EPICS values.	The current and voltage readings change accordingly to the load change.	☐ Pass ☐ Fail ☐ Retest
ST-06	Local Display	Monitor voltage, current, rail identification and status of all channels.	All necessary information is visible and correlates with the connected rail.	☐ Pass ☐ Fail ☐ Retest

ID	Test	Method	Pass Criterion	Result
ST-07	EPICS Communications	Run the IOC and check all voltage, current and status process variables.	All necessary process variables are connected and updated properly.	☐ Pass ☐ Fail ☐ Retest
ST-08	Limit / Alarm Operation	Set a limit and change the current of the electronic load or supply until the measured value reaches this limit.	The proper limit indication is created and then is cleared when the value is inside the limits.	☐ Pass ☐ Fail ☐ Retest
ST-09	Persistent Settings	Change some settings, cycle system power and check the value after the power-up.	The stored value is restored properly.	☐ Pass ☐ Fail ☐ Retest
ST-10	Communication Recovery	Disconnect Ethernet while the system is running and reconnect the network and monitor the IOC.	Monitoring on the local machine continues and communication recovers.	☐ Pass ☐ Fail ☐ Retest
ST-11	Repeatability	Make repeated measurements at selected points, including after power cycle.	Measurement remains within the specified acceptance limits with no intermittent failures of the channels.	☐ Pass ☐ Fail ☐ Retest
ST-12	End-To-End Demonstration	Run all parts of the system and demonstrate the operation of local and remote monitoring by changing the load on the selected channel.	All hardware, firmware, display, network, IOC and test software run cooperatively with no system level failure.	☐ Pass ☐ Fail ☐ Retest

10.1.7 Detailed Test Procedures

10.1.7.1 ST-01: Power-On and Initialization

1. Inspect all power, sensor, display, and Ethernet connections.
2. Verify that the electronic load is disabled.
3. Energize the required monitored power supplies.
4. Apply power to the Arduino and monitoring electronics.
5. Observe the Arduino startup sequence.
6. Confirm that the GIGA Display initializes.
7. Confirm that all six expected rail channels are displayed.
8. Start or verify operation of the EPICS IOC.
9. Verify that the IOC establishes communication with the monitor.
10. Record any startup errors or unexpected behavior.

The test passes when the complete system reaches its normal operating state without manual recovery, all six rail channels are present, and network communication can be established.

10.1.7.2 ST-03 and ST-04: Voltage and Current Measurement

Voltage and current accuracy testing will be performed for each of the six rails. Testing will be automated where practical to improve repeatability and preserve the raw test data.

For each selected rail:

1. Select the rail to be tested in the Python test program.
2. Verify the physical connection of the rail and electronic load.
3. Connect the DMM directly across the rail.
4. Set the electronic load to its initial test condition with the input disabled.
5. Enable the load.
6. Allow the readings to stabilize.
7. Record the DMM or E-Load reference voltage.
8. Record the electronic-load reference current.

9. Acquire the corresponding EPICS voltage and current values using PyEPICS.
10. Calculate the voltage and current measurement errors.
11. Repeat the measurement at several loads distributed throughout the intended operating range of the rail.
12. Disable the load before changing connections.
13. Save all raw measurements to the test-data file.

Voltage error will be calculated as

$$\text{Voltage Error (\%)} = \frac{V_{\text{monitor}} - V_{\text{reference}}}{|V_{\text{reference}}|} \times 100. \tag{10.1}$$

Current error will be calculated as

$$\text{Current Error (\%)} = \frac{I_{\text{monitor}} - I_{\text{reference}}}{|I_{\text{reference}}|} \times 100. \tag{10.2}$$

Absolute error will also be retained in the raw data. This is particularly important for low-current measurements where percentage error alone can become misleading.

A rail passes when each required measurement point remains within the specified acceptance criteria. If a point fails, the rail may be retested after the cause of the failure is investigated. Original failed data will not be deleted when a retest is performed.

10.1.7.3 ST-07: EPICS Communication Test

1. Power the complete monitor and connect it to the Ethernet network.
2. Start the EPICS IOC.
3. Verify that the IOC connects to the Arduino TCP server.
4. Observe the voltage and current process variables for all six rails.
5. Compare the EPICS values with the local display.
6. Change the load on a selected rail.
7. Verify that the corresponding EPICS voltage and current values update.
8. Repeat the load change on at least one positive and one negative rail.

The test passes when the required process variables remain connected, values change in response to the physical system, and EPICS and local-display values agree within their displayed resolution.

10.1.7.4 ST-08: Limit and Status Test

1. Select a rail for testing.
2. Record the existing voltage and current limit settings.
3. Configure a test limit that can be crossed safely without exceeding the electrical rating of the hardware.
4. Operate the rail below the limit and verify that normal status is displayed.
5. Increase or decrease the applied load until the selected threshold is crossed.
6. Verify the local status indication.
7. Verify the corresponding EPICS status.
8. Return the measurement to the normal operating range.
9. Verify that the status returns to normal.
10. Restore the original limit setting.

No test limit shall require the monitored supply, sensor, wiring, or electronic load to be operated outside its rated limits.

10.1.7.5 ST-09: Persistent Settings Test

1. Change one or more user-adjustable system settings to known values.
2. Verify that the settings are accepted by the firmware.
3. Remove power from the Arduino and monitor electronics.
4. Wait several seconds.
5. Restore power.
6. Allow the firmware to complete initialization.
7. Read the previously modified settings.
8. Compare the restored values with the values stored before power removal.

The test passes when all settings intended to be persistent are restored without requiring Ethernet communication or manual re-entry.

10.1.7.6 ST-10: Communication-Loss and Recovery Test

1. Establish normal communication between the monitor and IOC.
2. Verify that EPICS values are updating.
3. Disconnect the Ethernet cable.
4. Verify that local monitoring and the GIGA Display continue operating.
5. Observe the IOC response to the communication failure.
6. Reconnect the Ethernet cable.
7. Verify that communication is re-established.
8. Confirm that the process variables resume updating with valid values.

The system passes if the loss of Ethernet communication does not prevent local operation and normal EPICS communications resume after the network connection is restored.

10.1.8 Automated Test Data Collection

A Python test application will be used for measurement testing. The application interfaces with the Rigol DL3021 programmable electronic load and the EPICS IOC. EPICS Channel Access is performed using the PyEPICS library.

The test operator may select an individual power rail instead of being required to test all six rails in a single session. This permits channels to be tested or retested at different times while retaining all results as part of the same system test campaign.

For each test point, the following information will be retained when applicable:

- test date and time;
- rail identification;
- test iteration;
- electronic-load operating mode;
- programmed load value;
- reference voltage;
- reference current;
- EPICS voltage measurement;
- EPICS current measurement;

- absolute voltage error;
- percentage voltage error;
- absolute current error;
- percentage current error;
- applicable limits;
- test result; and
- operator notes.

Raw data will be preserved independently from the final report so that the report can be regenerated without repeating the physical test. After the required tests have been completed, the saved data will be processed using ReportLab to produce a final test report containing the recorded values and pass/fail results.

10.1.9 Test Execution and Result Classification

Each test will be assigned one of the following results:

PASS: The test was completed and all applicable acceptance criteria were met.

FAIL: The test was completed but one or more acceptance criteria were not met.

INCOMPLETE: The test could not be completed because of an unrelated equipment, software, or setup problem.

RETEST: A previously failed or incomplete test was repeated following a corrective action.

A failed test will not be deleted or overwritten following corrective action. The original result and subsequent retest will both be retained to provide traceability of the development process.

10.1.10 Suspension and Resumption Criteria

Testing will be immediately suspended if any of the following conditions occur:

- unexpected overheating of a component or conductor;
- evidence of a short circuit or incorrect wiring;
- operation outside the rated voltage or current of the test equipment;

- unstable or uncontrolled power-supply output;
- a firmware condition that could apply an unintended load;
- loss of isolation where isolation is required; or
- any condition that could damage the prototype or test equipment.

Testing may resume after the cause has been identified, corrective action has been completed, and the affected portion of the system has been inspected or verified before power is reapplied.

10.1.11 Test Responsibilities

As the designer and developer of the capstone project, Mike Capotosto will serve as the primary test operator. Responsibilities include:

- preparing and inspecting the test setup;
- configuring the test equipment;
- executing each test procedure;
- recording results;
- identifying and documenting failures;
- making required corrections;
- performing retests;
- maintaining the raw test data; and
- preparing the final test report and demonstration.

Where practical, automated data acquisition will be used to reduce transcription errors and improve repeatability.

10.1.12 Testing Risks and Contingencies

Several risks may affect system testing. Improper electronic-load connections, particularly when testing the negative supply rails, could damage the electronic load or power supply. Connections will therefore be verified before the load input is enabled.

Network problems could also produce an apparent measurement failure when the actual sensor hardware is operating correctly. Local display values and direct instrument measurements will be used to distinguish a network or IOC problem from a hardware measurement problem.

A failed sensor channel could prevent the complete six-channel system test from being performed. Because the test software allows individual rail selection, completed channel data can be retained while the affected channel is repaired and subsequently retested.

Finally, automatic test software itself could introduce an error in the recorded results. Critical results will therefore be compared with the front-panel readings of the electronic load, local display, and EPICS interface during initial testing to confirm correct operation of the automation.

10.1.13 Test Deliverables

The completed system test will produce the following deliverables:

- this system test plan;
- saved raw test data;
- automatically generated test report;
- documentation of failed tests and corrective actions;
- results of any required retests;
- photographs of the final test configuration; and
- video footage demonstrating selected system-level tests.

10.1.14 Test Completion Criteria

System-level testing will be considered complete when:

1. every test identified in Table 10.3 has been executed;
2. each of the six rail channels has completed voltage and current verification;
3. all critical functional tests have received a PASS result;
4. any failed tests have been documented and either corrected and retested or identified as known limitations;
5. raw test data has been preserved;
6. the final test report has been generated

Successful completion of this test plan will demonstrate that the individual hardware and software portions developed throughout the project operate together as a complete power supply monitoring system.

10.1.15 Test Reports

Refer to the Appendix D test plan for an example test report.

Chapter 11

Improvements and Optimizations to Solution

Despite the fact that the completed system satisfied the acceptance criteria, some improvements are recommended for the future PCB revision.

The most important hardware improvement is the incorporation of the SDA polarity fix directly in the PCB design. The prototype required the SN74LVC2G04 inverter to correct the SDA signal polarity after the originally installed LTC2945 device variant turned out to be incompatible with the intended isolation interface. The future revision should use the correct LTC2945 variant throughout the design or incorporate the required inversion directly in the schematic and PCB.

All required I^2C pull-up resistors and defined logic states should also be incorporated directly in the next PCB revision. It will eliminate the prototype modification and improve the repeatability of assembly and startup.

The firmware initialization can also be made more tolerant of arbitrary power-up sequencing. During the development, the communication could be failed when the controller tried to initialize the sensor before the corresponding isolated rail circuitry was powered. The future firmware revision could periodically retry the unavailable sensors so that the communication is established automatically when the monitored rail is powered.

The per-channel measurement calibration is another potential improvement. The present system calculates the current using the configured current sense resistance. The measurement of the actual resistance or the determination of the calibration factor against the external reference will allow the individual gain correction to be stored for each channel. It will help to reduce the systematic errors caused by the shunt tolerance and allow tighter acceptance limits if the improved measurement accuracy is required.

The additional environmental characterization is also useful if the design is going to be developed beyond the capstone prototype. The testing at multiple ambient temperatures can be used to determine whether the temperature affects the measurement accuracy. EMC testing can determine the susceptibility to and generation of the electromagnetic interference.

The long duration testing can be used to detect the intermittent communication, thermal, network or firmware problems that might not occur during the shorter acceptance test.

The automated test system should be retained for the future revisions. Since the raw measurements are saved separately from the report generation, the future hardware or firmware revisions can be tested using the same acceptance procedure and direct comparison between the revisions is possible.

These recommendations represent the refinements in reliability, manufacturability, calibration and qualification and not the changes to the basic architecture. The successful final test proves that the overall system concept is suitable for the further development.

Chapter 12

Conclusions

12.0.1 Did the Project Meet the Requirements of the Scope?

The project fulfilled the major requirements of the original project scope.

The completed system provides six independent voltage and current monitoring channels corresponding to +5VA, +5VB, +5 V, −5 V, +15 V, and −15 V. The high side current sensing is used for each channel while the electrical isolation is preserved between independently referenced monitored supplies and the common controller.

The completed hardware includes the sensor interface, current sense resistors, isolation circuitry, Arduino GIGA controller interface, local display, controller power circuitry and Ethernet interface. The software implementation includes the embedded measurement acquisition, conversion to engineering units, configurable limits, alarm processing, persistent configuration storage, local graphical display, network communication, the EPICS IOC, process variable database records and the Phoebus operator interface.

The final system testing confirmed the correct operation of all six measurement channels, the correct positive and negative voltage representation, programmable limit operation, persistent settings, local display operation, Ethernet communication, EPICS monitoring, communication recovery, repeatability, and end-to-end operation.

The hardware, firmware, IOC files, operator interface files, documentation and other project artifacts were also maintained under revision control as required by the project scope.

Based on the completed implementation and the final system test results, the capstone project is therefore considered to have fulfilled its technical scope.

12.0.2 Was the Project Completed on Time?

The project was completed within the planned capstone schedule.

The original project plan distributed the development across the eight week ET410 planning phase and the following eight week ET450 implementation phase, with the completed system

expected to be ready for demonstration by Week 7 of ET450. The final automated system test was completed on October 18, 2026, giving the completed and verified system before the planned final demonstration period.

The project experienced the schedule risk during the hardware bring-up. Troubleshooting of the isolated I^2C interface required additional effort after the SDA polarity problem and other communication problems were discovered. The startup sequencing and Ethernet integration also required the troubleshooting that was not anticipated in the initial plan.

These problems consumed the schedule margin, but they were fixed without preventing the completion of the prototype and final system test within the capstone schedule. The project is therefore considered to have fulfilled the schedule requirement.

12.0.3 Was the Project Completed Under Budget?

The project was not completed within the original preliminary hardware budget.

The initial project proposal assumed the material and contingency budget of $660. This estimate was based on the incomplete information about the detailed component selection, custom PCB layout and final vendor pricing.

After completion of the detailed design and Bill of Materials, the estimated one system hardware purchase cost became $1,019.03. It includes $242.36 in the Sensor Board components, $473.57 for the minimum Sensor Board PCB order, $180.90 in the Arduino Shield components, and $122.19 for the minimum Arduino Shield PCB order.

Compared to the original $660 estimate, the final documented hardware purchase estimate is $359.03 higher than the preliminary budget. The additional cost is due to a better definition of the component as well as the minimum order cost related to the fabrication of the custom PCBs.

Thus, the cost estimate was good to use as a preliminary cost estimate value but did not provide enough cost to build a complete prototype of a custom PCB. The project should be considered above the original hardware budget.

This cost comparison concerns only the documented prototype hardware cost. Laboratory test equipment, software, labor, shipping, taxes, enclosure hardware, cabling and any other items not included into the project cost analysis are not included into this cost comparison.

12.0.4 Did the Quality and Performance Meet Stakeholder Expectations?

The quality and performance of the completed prototype met the expectations set for the capstone project.

The main project stakeholders are the project developer, the course instructor and the potential technical users of the system. The best evidence of system quality is the completed acceptance test. All twelve expected system tests were completed successfully, all six mon-

itored rails passed the voltage and current check, and the complete system demonstration was successful without any system failures.

The prototype showed the capability to collect isolated voltage and current measurements, display the data locally, compare the data with configurable limits, keep the configuration after the restart, send the data via Ethernet, support the EPICS integration, present the data using Phoebus and to restore the operation after the network connection interruption.

The project provided the possibility to develop the repeatable automated test procedure instead of using the subjective one. The raw data were saved and used for the creation of a test report including the measurement data, pass/fail result and the test evidence. It improves the confidence in the final result and allows assessing future revisions of the hardware.

The prototype still includes some features which should be improved in the future production revision of the design, such as SDA inverter modification, modified implementation of the I^2C pull-up, correct startup sequence and the possible implementation of the per-channel calibration. They do not prevent the prototype from meeting the acceptance criteria, but may influence the manufacturability and further improvements of the design.

Based on the final test results, the completed project can be stated to meet the stakeholder expectations in terms of functionality and performance.

12.0.5 Overall Project Assessment

The Multi-Rail Isolated Power Supply Monitoring System successfully passed the full project cycle including the project definition and preliminary design, custom hardware development, firmware implementation, network and EPICS integration, board bring-up, troubleshooting and system verification.

The biggest challenges of the project were related to the hardware integration instead of development of the individual functions of the system. The troubleshooting of the isolated I^2C interface showed the necessity to check the component version, polarity, pull-up requirement, signal states and startup behavior in the physical implementation. These issues were found by the measurements, fixed and retested in the final system tests.

The completed prototype met the defined project scope, completed in the required time and met the final technical acceptance criteria. Although the project cost exceeds the original preliminary hardware budget, the detailed cost estimation provides more realistic basis for the estimation of the cost of prototype and future production revision.

Finally, the successful completion of the system test showed that the hardware, embedded firmware, local user interface, Ethernet communication, EPICS IOC, Phoebus interface and the automated test software work as the integrated system. Thus, the project successfully reached the main goal of development and demonstration of the functioning six-channel isolated power supply monitoring system.

Chapter 13

References

Appendix A

Functional Specifications

The functional specifications used for the design are summarized in Table A.1. These values define the intended operating envelope of the prototype and establish the quantities that must be checked during circuit analysis and later verification testing.

Table A.1: Functional specifications.

Function	**Specification**
Monitored rails	+5 V A, +5 V B, +5 V, -5 V, +15 V, and -15 V
Number of channels	Six independent voltage/current measurement channels
+5 V A/B current range	0 to 2 A design range per rail
+5 V, -5 V, +15 V, -15 V current range	0 to 1 A design range per rail
Voltage measurement resolution	25 mV/LSB nominal using the LTC2945 rail-voltage ADC
Current-sense resolution	25 μV/LSB across the selected shunt resistor; approximately 0.641 mA/LSB with 39 mΩ and 0.500 mA/LSB with 50 mΩ
Isolation	Independent digital isolation for every floating monitor channel
Alarm processing	User-configurable upper/lower voltage and current limits with local and EPICS status indication
Fault handling	Detect failed I^2C transactions and prevent invalid data from being treated as a valid measurement
Local interface	GIGA Display Shield showing voltage, current, alarm state, communication state, and configured limits
Network interface	Ethernet PSC server for EPICS IOC communication plus browser-based configuration/monitoring services
Configuration retention	Network parameters and alarm limits stored in nonvolatile memory
Controller logic level	3.3 V; external interfaces must not exceed the GIGA I/O limits

Appendix B

Design Calculations and Circuit Analysis

This appendix contains the electrical calculations used to select and verify the preliminary hardware. In addition to nominal values, voltage, current, power, logic-level compatibility, and component capability are considered so that the design does not intentionally operate a component beyond its specified limits.

B.1 Current-Sense Shunt Resistor Sizing

The LTC2945 measures the differential voltage between SENSE+ and SENSE– with a full-scale range of 102.4 mV. The ideal shunt resistance for a specified maximum measurable current is therefore

$$R_{SHUNT,ideal} = \frac{V_{SENSE,FS}}{I_{MAX}} = \frac{102.4\,\text{mV}}{I_{MAX}}. \tag{B.1}$$

To prevent saturation, a scaling factor, K_H, is applied to leave some headroom:

$$R_{SHUNT} = K_H \frac{102.4\,\text{mV}}{I_{MAX}}, \qquad 0 < K_H < 1. \tag{B.2}$$

For example, $K_H = 0.80$ reserves 20% headroom for overload, tolerance, and transient conditions.

The shunt power at maximum continuous current is

$$P_{SHUNT} = I_{MAX}^2 R_{SHUNT}. \tag{B.3}$$

The installed resistor power rating should exceed this value.

B.1.1 Current Measurement Resolution

The LTC2945 current-sense ADC has a nominal sense-voltage resolution of 25 µV per LSB. The current represented by one ADC count is

$$I_{LSB} = \frac{25\,\mu V}{R_{SHUNT}}. \tag{B.4}$$

This relationship provides an important design tradeoff: a smaller shunt lowers power dissipation but also increases the minimum current increment represented by one ADC count.

B.1.2 Production Shunt Design Table

Table B.1: Production current-sense resistor calculations.

Rail	I_{MAX} (A)	R_{ideal} (Ω)	Selected R (Ω)	V_{SHUNT} (mV)	I_{LSB} (mA/LSB)	P_{MAX} (W)
+5VA	2.00	0.0512	0.039	78.0	0.641	0.156
+5VB	2.00	0.0512	0.039	78.0	0.641	0.156
+5V	1.00	0.1024	0.050	50.0	0.500	0.050
-5V	1.00	0.1024	0.050	50.0	0.500	0.050
+15V	1.00	0.1024	0.050	50.0	0.500	0.050
-15V	1.00	0.1024	0.050	50.0	0.500	0.050

B.1.3 Evaluation-Board Shunt Calculation

The prototype LTC2945 evaluation board was tested from a 5 V source with a load range of approximately 1 kΩ to 10 kΩ. This is a low-current bench-test condition and is separate from the production shunt design.

At the minimum load resistance, the shunt required to place the current-sense input at full scale is

$$R_{S,FS} = \frac{V_{SHUNT} R_L}{V_S - V_{SHUNT}} = \frac{(0.1024\,V)(1\,k\Omega)}{5\,V - 0.1024\,V} = 20.91\,\Omega. \tag{B.5}$$

Applying approximately 15% headroom gives

$$R_S = (0.85)(20.91\,\Omega) \approx 17.8\,\Omega. \tag{B.6}$$

At a 10 kΩ load,

$$I_{MIN} = \frac{5\,V}{10\,k\Omega + 17.8\,\Omega} \approx 499.1\,\mu A, \tag{B.7}$$

and therefore

$$V_{SHUNT,MIN} = I_{MIN} R_S \approx (499.1\,\mu\text{A})(17.8\,\Omega) \approx 8.88\,\text{mV}. \tag{B.8}$$

The prototype sense voltage therefore remains inside the LTC2945 102.4 mV full-scale range over the intended test loads.

B.2 PCB Trace and Copper-Pour Current Capacity

Power-path copper must carry the maximum rail current without excessive temperature rise or voltage drop. KiCad's track-width calculator uses a closed-form fit to IPC-2152 data. For an external trace it can be expressed as

$$\Delta T = K I^a W^b T_h^c, \tag{B.9}$$

where I is current in amperes, W is trace width in mils, T_h is copper thickness in mils, and for external tracks the KiCad coefficients are

$$K = 215.3, \qquad a = 2, \qquad b = -1.15, \qquad c = -1.0. \tag{B.10}$$

Solving for minimum width,

$$W_{MIN} = \left(\frac{\Delta T}{K I^a T_h^c} \right)^{1/b}. \tag{B.11}$$

For 1 oz copper ($T_h \approx 1.378$ mil) and a preliminary 10 °C allowed rise, the calculated external-trace widths are approximately 3.27 mil at 0.5 A, 10.9 mil at 1 A, and 36.4 mil at 2 A. These are thermal minimums only; the actual layout should use wider traces or copper pours where space permits.

B.3 I²C Pull-Up Resistor Sizing

The I2C bus buffer and I2C line driver both have internal active current sources that eliminate the need for external pullups. Footprints were placed for 4.7k pullups but are set as Do Not Populate.

B.4 LTC2945 Voltage Measurement Resolution

The LTC2945 rail-voltage channel has a nominal full-scale range of 102.4 V and a nominal resolution of 25 mV per LSB. The quantization-only percentage of a measured rail voltage is therefore

$$\epsilon_Q = \frac{25\,\text{mV}}{V_{RAIL}} \times 100\%. \tag{B.12}$$

For example, one count corresponds to approximately 0.50% of a 5 V rail and 0.17% of a 15 V rail. This is only ADC resolution and not total measurement accuracy.

B.5 Negative-Rail Sign Convention

The -5 V and -15 V supplies are electrically isolated from the controller. The monitor can therefore be placed in the high side of each isolated rail and read a positive local differential quantity even when the system-level rail is labeled negative. Firmware applies the configured sign after acquisition:

$$V_{DISPLAY} = S_V V_{MEASURED}, \qquad S_V \in \{+1, -1\}. \tag{B.13}$$

The same channel metadata is used by the local display and network interface so that a negative rail is represented consistently throughout the system.

B.6 Alarm-Limit Scaling

User alarm thresholds are stored and compared in engineering units. For a lower voltage limit V_L and upper voltage limit V_H, the normal voltage state is

$$V_L \le V_{MEAS} \le V_H. \tag{B.14}$$

Similarly, for lower and upper current limits,

$$I_L \le I_{MEAS} \le I_H. \tag{B.15}$$

If raw ADC thresholds are ever written directly to the LTC2945, the corresponding register value must be calculated from the ADC LSB size. For the current-sense channel,

$$N_I = \frac{I_{LIMIT} R_{SHUNT}}{25\,\mu\text{V/LSB}}, \tag{B.16}$$

and for the rail-voltage channel,

$$N_V = \frac{V_{LIMIT}}{25\,\text{mV/LSB}}. \tag{B.17}$$

B.7 Controller-Supply Power Budget

A preliminary controller power budget was developed using datasheet maximum values where available and conservative design-current allowances for assemblies where a complete board-level maximum current is not specified. A 25% design margin is applied to the calculated load.

Table B.2: Preliminary controller-side power budget.

Load	Supply (V)	Design Current (A)	Power (W)
Arduino GIGA R1 WiFi	5.0	0.500	2.500
GIGA Display Shield	3.3	0.250	0.825
W6100 Ethernet interface	3.3	0.265	0.875
I^2C isolation/controller circuitry	3.3	0.065	0.215
Subtotal	–	–	4.415
25% design margin	–	–	1.104
Total	–	–	**5.519**

B.8 Component Capability Checks

The calculations above establish the required electrical values. The following checks compare those values with the capability of the selected components.

B.9 Current-Sense Resistor Power Margin

The selected LVK-series 39 mΩ and 50 mΩ four-terminal shunts are rated at 0.75 W. At the maximum design current of 2 A, the 39 mΩ resistor dissipates

$$P = (2\,\mathrm{A})^2(0.039\,\Omega) = 0.156\,\mathrm{W}. \tag{B.18}$$

This is approximately 20.8% of the 0.75 W rating, corresponding to about a 4.8:1 power-rating margin before considering PCB temperature and derating. At 1 A, a 50 mΩ shunt dissipates only 0.050 W, approximately 6.7% of a 0.75 W rating.

B.10 LTC2945 Input and ADC Range

The highest monitored nominal rail is 15 V, far below the LTC2945 0–80 V rail monitoring range. The selected production shunts produce 78 mV at 2 A on the 39 mΩ channels and 50 mV at 1 A on the 50 mΩ channels. Both values are below the 102.4 mV current-sense full scale, leaving approximately 24.4 mV and 52.4 mV of ADC headroom respectively. The LTC2945 can operate from an external 2.7–5.9 V supply, so a 3.3 V sensor-side logic supply is within its specified operating range.

B.11 Microcontroller I/O Compatibility

The Arduino GIGA R1 WiFi operates with 3.3 V logic and specifies 8 mA maximum DC current per I/O pin. The controller-side I^2C interface is a logic interface, not a load-power interface. The TCA9803 used in the design supports a 0.8–3.6 V A-side supply and a 1.65–3.6 V B-side supply and provides an internal B-side current source of up to 3 mA. A 3 mA bus-low current is below the 8 mA GIGA per-pin limit, providing more than a factor of two current margin. The GIGA pins therefore do not directly source the optocoupler LED or rail-load current; the external interface circuitry performs the required buffering and isolation.

B.12 Digital-Isolation Margin

The ACPL-064L digital optocoupler is specified for 3.3 V/5 V operation and is UL 1577 recognized for 3750 V_{RMS} isolation for one minute. The largest normal potential difference between the nominal monitored rails is only 30 V from -15 V to +15 V. The optocoupler rating therefore greatly exceeds the functional isolation voltage required by this low-voltage monitoring system. This comparison verifies component capability; it is not intended to redefine the safety rating of the complete assembled product, which also depends on PCB creepage, clearance, connectors, wiring, and enclosure construction.

B.13 3.3 V Regulator Loading

The TPS62912 is rated for up to 2 A output. Using the Week 1 design-current allowances for the display, Ethernet interface, and controller-side isolation circuitry gives an estimated 3.3 V load of

$$I_{3.3} = 0.250 + 0.265 + 0.065 = 0.580\,\text{A}. \tag{B.19}$$

This is approximately 29% of the regulator's 2 A output rating and leaves substantial current margin for transient and implementation variation.

B.14 Tolerance and Measurement-Uncertainty Considerations

The production current-sense resistors have a 1% resistance tolerance. Because current is calculated from $I = V_{SHUNT}/R_{SHUNT}$, an uncorrected 1% shunt error produces approximately a 1% gain error in calculated current. On a 2 A range, this corresponds to approximately 20 mA, which is much larger than the 0.641 mA quantization step of the 39 mΩ channel. The LTC2945 also has its own conversion error; therefore component tolerance rather than ADC quantization is an important contributor to absolute current accuracy. If higher accuracy is

required, the actual shunt resistance can be measured and a per-channel calibration factor stored in firmware.

A conservative worst-case linear addition of a 1% shunt tolerance and the LTC2945's less-than-±0.75% total unadjusted conversion error gives an upper-bound estimate of approximately ±1.75% before wiring, thermal, and calibration effects. This estimate is used for design margin rather than as a claim of final calibrated system accuracy.

B.15 Reference Specifications Used for Capability Checks

- Analog Devices, LTC2945 data sheet and product information: `https://www.analog.com/en/products/ltc2945.html`
- Arduino, GIGA R1 WiFi technical specifications: `https://store.arduino.cc/products/giga-r1-wifi`
- Broadcom, ACPL-064L low-power digital optocoupler data sheet/product page: `https://www.broadcom.com/products/optocouplers/industrial-plastic/digital-optocouplers/10mbd/acpl-064l-000e`
- Texas Instruments, TCA9803 I^2C buffer data sheet: `https://www.ti.com/product/TCA9803`
- Texas Instruments, TPS62912 buck converter data sheet: `https://www.ti.com/product/TPS62912`
- Ohmite LVK-series current-sense resistor specifications: `https://www.ohmite.com/assets/files/Datasheets/lvk-series.pdf`

Appendix C

Design Documents

C.1 Schematics

C.2 Software Flow Chart

This appendix provides the detailed software flow requested for the Week 2 preliminary-design package. It shows the firmware acquisition/alarm loop and the PSC-to-EPICS data path used by the implemented software.

The software for the Multi-Rail Isolated Power Supply Monitoring System is maintained with the hardware design files in the project Git repository:

`https://github.com/capotosto/PS_Monitor`

The repository provides version control for the Arduino GIGA firmware, hardware-design files, and EPICS IOC software used by the monitoring system. The embedded firmware is maintained under the `firmware` portion of the repository, while the EPICS IOC and associated database/configuration files are maintained under `epics-ioc`. The flowcharts in this section describe the major software operations implemented by these portions of the project.

The Arduino GIGA firmware performs system initialization, acquires voltage and current measurements from the six LTC2945 monitor channels, evaluates alarm conditions, updates the local display, and publishes data through the Ethernet interface. The primary firmware execution sequence is:

1. Initialize the GIGA Display, persistent settings, Ethernet interface, I^2C interface, and channel configuration.
2. Load stored network parameters and voltage/current alarm limits from persistent storage.
3. Initialize communication with each of the six LTC2945 monitoring channels.
4. Enter the continuous acquisition and communication loop.
5. Poll each LTC2945 channel for voltage and current measurements.
6. Verify I^2C communication status for each channel.
7. If communication fails, mark the associated channel with an I^2C error and prevent invalid measurement data from being used.
8. Convert valid raw measurement data to engineering units and apply the appropriate channel polarity and calibration/scaling information.
9. Compare each voltage and current measurement with its configured upper and lower alarm limits.
10. Update the GIGA Display when measurement values, alarm states, or communication states change.

11. Publish the current measurements and status information through the Ethernet PSC protocol interface.
12. Service incoming configuration requests and store modified settings when required.
13. Repeat the acquisition and communication sequence continuously.

C.2.1 Arduino GIGA Firmware Flowchart

Figure C.1 shows the top-level execution flow of the Arduino GIGA firmware. Following initialization, the controller continuously acquires data from each monitoring channel, verifies communication, converts valid measurements to engineering units, evaluates alarm limits, updates the local display, and services the Ethernet/PSC interface.

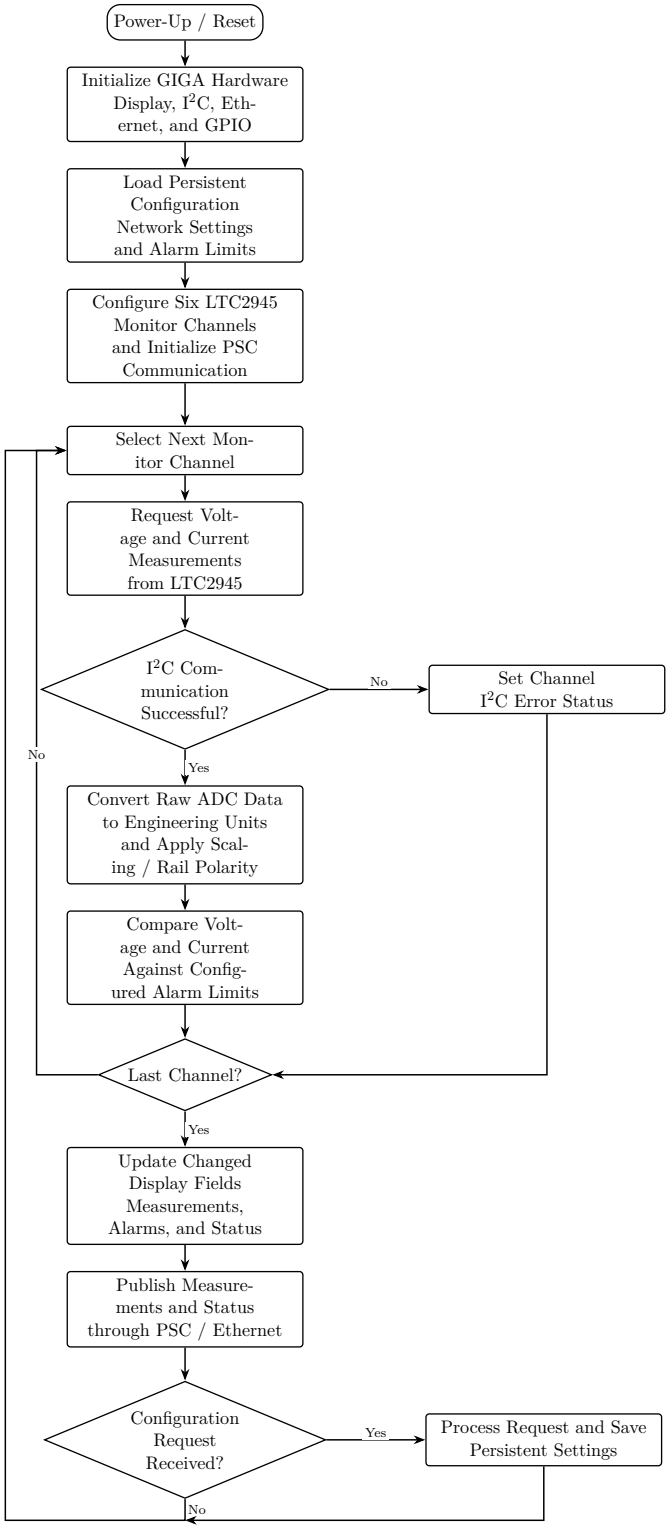


Figure C.1: Top-level Arduino GIGA firmware flowchart. Following initialization, the controller continuously acquires measurements from the six LTC2945 monitoring channels, validates I^2C communication, converts valid data to engineering units, evaluates configured alarm limits, updates the local display, publishes measurements through the PSC/Ethernet interface, and services configuration requests.

C.2.2 PSC and EPICS Communication Flow

The Arduino GIGA does not directly implement EPICS Channel Access. Instead, measurement and status information is transmitted through the Ethernet interface using the PSC protocol. The EPICS IOC uses PSCDriver to communicate with the GIGA, maps the received values into EPICS process variables, and makes those process variables available to the CS-Studio Phoebus operator interface.

This separation allows the embedded controller to remain responsible for hardware acquisition and local alarm evaluation while the IOC provides the interface between the embedded hardware and the facility control system. Figure C.2 summarizes this software data path.

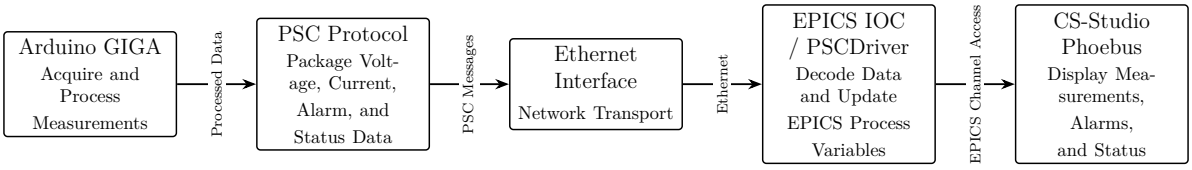


Figure C.2: Software communication path from the Arduino GIGA firmware through PSCDriver and the EPICS IOC to the CS-Studio Phoebus operator interface.

C.2.3 Software Configuration Management

Firmware, IOC configuration files, database definitions, and operator-interface files are maintained under Git version control so that hardware and software revisions can be associated with a defined project baseline. The current software implementation and revision history are available in the `PS_Monitor` repository at:

`https://github.com/capotosto/PS_Monitor`

Changes made during implementation and testing are committed to the repository, providing traceability between the preliminary design presented in this report and the final implemented system. The repository therefore serves as the controlled source for the firmware and control-system software associated with the project.

C.3 Source Code